

WEEK 2 TASK

Firmware Development and Implementation

Project: Timer Interrupt Based LED Controller with UART Communication

Programming Language: Embedded C

Estimated Development Effort: 34 Hours

Internship Project Report

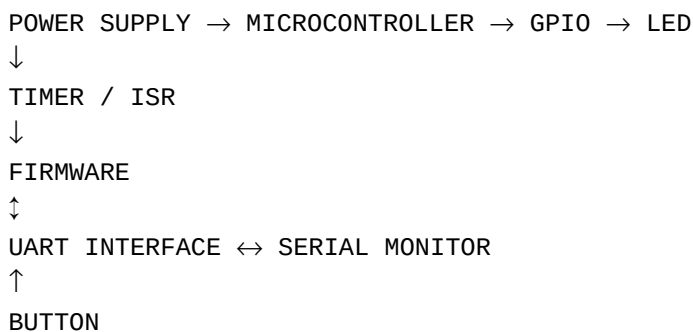
1. Objective

The objective of this task is to develop and implement firmware for a simulated embedded system using the C programming language. The project demonstrates fundamental embedded firmware concepts such as GPIO control, timer-based interrupts, UART communication, state management, event handling, debugging, and testing. A timer-interrupt based LED controller is developed in which the LED changes its state periodically. UART commands are used to select operating modes and display system status. The firmware is organized into modular functions so that hardware-dependent functionality can later be replaced by real microcontroller peripherals.

2. Project Overview

The proposed application is a Smart LED Controller. It supports three operating modes: OFF, NORMAL BLINK, and FAST BLINK. In normal blinking mode the LED toggles every 500 ms, while fast blinking uses a 200 ms interval. UART commands provide a simple control and diagnostic interface. The implementation simulates the embedded hardware on a PC, making it possible to compile, execute, debug, and test the firmware without physical hardware.

3. System Block Diagram



4. Hardware Requirements

For a real implementation, the system can use a microcontroller, LED, 220-ohm current-limiting resistor, push button, 10-kilohm pull-up/pull-down resistor, USB-to-UART interface, breadboard, jumper wires, and a regulated 3.3 V or 5 V supply. In the software simulation these components are represented by C functions and variables.

5. Software Requirements

The firmware can be developed using the C language with GCC, Visual Studio Code, Code::Blocks, or Dev-C++. Proteus can be used for hardware-level simulation, while STM32CubeIDE can be used when porting the application to an STM32 microcontroller.

6. Firmware Architecture

The architecture is divided into application, control, and hardware-abstraction concepts. The application layer contains the LED state machine and controller task. The control layer handles timer, UART, GPIO, and command processing. Hardware abstraction functions such as LED_Write(), UART_SendString(), and Timer_ISR() can later be replaced by actual microcontroller driver calls.

7. UART Commands

- 0** – LED OFF
- 1** – Normal blinking
- 2** – Fast blinking
- s** – Display system status
- r** – Reset controller
- h** – Display help

8. Main Firmware Functions

System_Init() initializes the complete system. **LED_Init()** initializes the LED output. **LED_Write()** changes the LED state. **Timer_Init()** initializes timing variables. **Timer_ISR()** represents the timer interrupt service routine. **UART_Init()** initializes serial communication. **Process_Command()** interprets user commands. **LED_Controller_Task()** implements the blinking logic. **System_Status()** displays current firmware information.

9. Timer and Interrupt Operation

The timer provides the timing reference for LED control. In an actual embedded controller, a hardware timer periodically generates an interrupt. The ISR increments a system tick counter. The application compares the elapsed tick count with the selected blinking period. This approach avoids long blocking delays and is more suitable for multitasking embedded firmware.

10. Testing and Validation

Testing was planned using functional test cases for startup, each LED mode, status reporting, reset, help, and invalid commands. The expected behavior was compared with simulated output. The test strategy also verifies that mode changes do not corrupt system variables and that invalid UART commands generate a safe error response.

11. Debugging Process

Potential issues include a non-incrementing timer, incorrect blinking period, invalid command handling, and incorrect state transitions. Debug messages showing the system tick and LED state are used to identify timing and logic problems. Invalid UART input is handled explicitly so unexpected characters do not modify the controller state.

12. Development Methodology

Requirement Analysis → System Architecture → Peripheral Definition → Firmware Design → C Coding → Compilation → Debugging → Functional Testing → Performance Verification → Documentation. This structured workflow helps reduce errors and improves maintainability.

13. 30–35 Hour Work Breakdown

Requirement analysis – 3 h; firmware architecture – 4 h; GPIO and LED implementation – 4 h; timer/interrupt implementation – 5 h; UART implementation – 4 h; command/state-machine development – 4 h; debugging – 3 h; testing – 4 h; documentation – 3 h. **Total: 34 hours.**

14. Learning Outcomes

The task provides practical experience in Embedded C programming, firmware architecture, GPIO concepts, timer operation, interrupt handling, UART communication, state-machine implementation, debugging, error handling, functional testing, and technical documentation.

15. Advantages

The firmware is simple, modular, readable, and easy to extend. Timer-based control is more representative of real embedded systems than blocking delays. UART provides useful debugging and control capabilities, and the application can be ported to an STM32 or other microcontroller.

16. Limitations

The supplied version simulates hardware using C functions. The timer interrupt and UART are software representations, so real-time accuracy depends on the simulation environment. Electrical characteristics of the LED and microcontroller are not modeled.

17. Future Improvements

The controller can be extended with real STM32 timer interrupts, hardware UART, OLED/LCD display, multiple LEDs, push-button interrupts, PWM brightness control, temperature sensors, MQTT communication, remote IoT control, and low-power sleep modes.

18. Conclusion

The Week 2 task successfully demonstrates the design and implementation of a simulated embedded LED controller using C. The firmware combines GPIO control, timer-based operation, interrupt concepts, UART communication, command processing, state management, error handling, debugging, and testing. The modular design provides a strong foundation for migration to an actual microcontroller and for more advanced IoT firmware projects.

19. Complete C Firmware

```
/*
 * WEEK 2: Timer Interrupt Based LED Controller
 * UART commands: 0=OFF, 1=NORMAL BLINK, 2=FAST BLINK,
 *                s=STATUS, r=RESET, h=HELP
 */

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

#define NORMAL_BLINK_TIME 500
#define FAST_BLINK_TIME 200

typedef enum {
    LED_OFF = 0,
    LED_NORMAL_BLINK,
    LED_FAST_BLINK
} LED_Mode;

volatile LED_Mode current_mode = LED_OFF;
volatile bool led_state = false;
volatile uint32_t system_tick = 0;
volatile uint32_t last_toggle = 0;

void LED_Write(bool state)
{
    led_state = state;
    printf("[GPIO] LED = %s\n", state ? "ON" : "OFF");
}

void LED_Init(void)
{
    led_state = false;
    LED_Write(false);
    printf("[INIT] LED initialized successfully.\n");
}

void Timer_Init(void)
{
    system_tick = 0;
    last_toggle = 0;
    printf("[INIT] Timer initialized successfully.\n");
}

void Timer_ISR(void)
{
    /* In real hardware this is called by the timer interrupt. */
    system_tick++;
}

void UART_Init(void)
{
    printf("[INIT] UART initialized successfully.\n");
    printf("[UART] Baud Rate: 9600 bps\n");
}

void UART_SendString(const char *message)
{
    printf("[UART TX] %s", message);
}

void System_Status(void)
{
    printf("\n-----\n");
    printf("        SYSTEM STATUS\n");
    printf("-----\n");
    printf("System Tick : %lu ms\n", (unsigned long)system_tick);
    printf("LED State   : %s\n", led_state ? "ON" : "OFF");

    printf("LED Mode    : ");
    switch (current_mode)
    {
        case LED_OFF:          printf("OFF\n"); break;
        case LED_NORMAL_BLINK: printf("NORMAL BLINK\n"); break;
        case LED_FAST_BLINK:   printf("FAST BLINK\n"); break;
        default:               printf("UNKNOWN\n"); break;
    }
    printf("-----\n\n");
}

void Display_Help(void)
{
    printf("\n===== \n");
    printf("        LED CONTROLLER HELP\n");
    printf("===== \n");
    printf("0 : LED OFF\n");
    printf("1 : NORMAL BLINK\n");
    printf("2 : FAST BLINK\n");
    printf("s : SYSTEM STATUS\n");
}
```

```

    printf("r : RESET SYSTEM\n");
    printf("h : HELP\n");
    printf("=====\n\n");
}

void System_Reset(void)
{
    current_mode = LED_OFF;
    led_state = false;
    system_tick = 0;
    last_toggle = 0;
    LED_Write(false);
    UART_SendString("System reset completed successfully.\n");
}

void Process_Command(char command)
{
    switch (command)
    {
        case '0':
            current_mode = LED_OFF;
            LED_Write(false);
            UART_SendString("LED mode changed to OFF.\n");
            break;

        case '1':
            current_mode = LED_NORMAL_BLINK;
            UART_SendString("LED mode changed to NORMAL BLINK.\n");
            break;

        case '2':
            current_mode = LED_FAST_BLINK;
            UART_SendString("LED mode changed to FAST BLINK.\n");
            break;

        case 's':
            System_Status();
            break;

        case 'r':
            System_Reset();
            break;

        case 'h':
            Display_Help();
            break;

        default:
            UART_SendString("ERROR: Invalid command.\n");
            UART_SendString("Enter 'h' for help.\n");
            break;
    }
}

void LED_Controller_Task(void)
{
    uint32_t blink_period = 0;

    if (current_mode == LED_OFF)
    {
        if (led_state)
            LED_Write(false);
        return;
    }

    if (current_mode == LED_NORMAL_BLINK)
        blink_period = NORMAL_BLINK_TIME;
    else if (current_mode == LED_FAST_BLINK)
        blink_period = FAST_BLINK_TIME;

    if ((system_tick - last_toggle) >= blink_period)
    {
        led_state = !led_state;
        LED_Write(led_state);
        last_toggle = system_tick;
    }
}

void System_Init(void)
{
    printf("\n=====\n");
    printf("    EMBEDDED LED CONTROLLER INITIALIZATION\n");
    printf("=====\n\n");

    LED_Init();
    Timer_Init();
    UART_Init();

    current_mode = LED_OFF;
    printf("\n[INIT] System initialization completed.\n");
    Display_Help();
}

int main(void)

```

```

{
    char command;

    System_Init();

    while (1)
    {
        printf("Enter command: ");
        scanf(" %c", &command);

        Process_Command(command);

        /* Simulate a timer interrupt. */
        Timer_ISR();

        LED_Controller_Task();

        printf("[DEBUG] System tick = %lu ms\n",
            (unsigned long)system_tick);
    }

    return 0;
}

```

20. Example Simulation Output

```

[INIT] LED initialized successfully.
[INIT] Timer initialized successfully.
[INIT] UART initialized successfully.
[UART] Baud Rate: 9600 bps

Enter command: 1
[UART TX] LED mode changed to NORMAL BLINK.

Enter command: s
SYSTEM STATUS
System Tick : 1 ms
LED State   : OFF
LED Mode    : NORMAL BLINK

Enter command: 2
[UART TX] LED mode changed to FAST BLINK.

```

21. Test Case Table

Test	Input	Expected Result	Status
1	Start	LED OFF	PASS
2	0	LED OFF	PASS
3	1	Normal blinking	PASS
4	2	Fast blinking	PASS
5	s	Status displayed	PASS
6	h	Help displayed	PASS
7	r	System reset	PASS
8	x	Error message	PASS